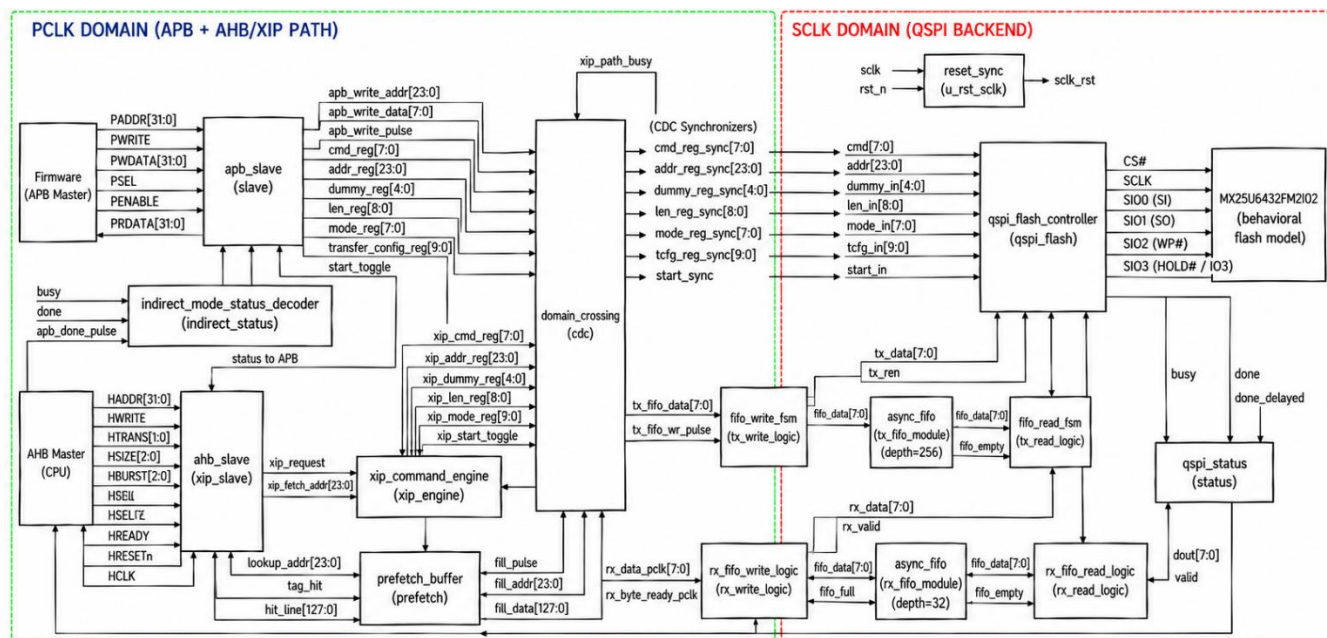


PROJECT – 5

QSPI Flash Controller with Execute-In-Place (XIP)

- G Gokul (EMP ID: 6892)

MICRO ARCHITECTURE



The entire project can be divided into two path, with each path trying to access the flash via the QSPI Flash Controller. The paths are:

1. **Indirect mode(APB):** Indirect mode is nothing but a generic command transfer engine. In indirect mode, Firmware feeds the input for APB slave. The firmware fills the register bank one by one and then starts the data transmission by enabling start flag in control status register present in the register bank. Once the transmission has started, the register data is sent to “domain_crossing” module, which then checks if the flash is busy and send performs enable signal based data crossing based on priority with “qspi_flash_controller” module via a toggle synchronizer since it is fast to slow domain crossing. The Flash Controller latches these values and sent them to the Behavioral Flash Model via the serial pins.
2. **XIP Mode:** In XIP mode, unlike Indirect mode, only address is given as input. The AHB Slave first checks if the prefetch buffer has the requested address data stored in it. If the data is present, then it is fetched and given back to AHB Master, else AHB slave has to access the Flash to read data.

SPECIFICATION

Target device: Macronix MX25U6432FM2I02 (QSPI NOR flash)

Top module: top_cdc_fifo_integrated.v

Clock domains: AMBA domain (hclk = pclk = 50 MHz, unified) and SCLK domain (12.5 MHz, 4:1 ratio, free-running/independent)

Testbench: tb_5.v (self-checking, locked command-set coverage)

BIG PICTURE

There are two ways to talk to the flash:

1. **Indirect/APB path** - firmware programs command/address/length/transfer-config registers over APB and pulses a start bit. Used for WREN, PP, SE, BE, WRSR, RDID, RDSR, and all the "manual" read variants (FAST_READ/QOR/DOR/DIO/QIOR).
2. **XIP/AHB path** - a CPU fetching code/data over AHB-Lite gets served out of a 16-byte prefetch line cache. On a cache miss, an autonomous hardware engine (xip_command_engine) issues an RDSR poll (wait for WIP=0) followed by a QIOR burst to refill the line, with no firmware involvement.

Both paths share one physical QSPI engine (qspi_flash_controller + one TX FIFO + one RX FIFO). An ownership mux in top_cdc_fifo_integrated selects, cycle by cycle, whether the shared engine's inputs come from the APB register bank or from the XIP engine's internal register bank (xip_path_busy is the select). Arbitration rule: APB wins on conflict — ahb_slave holds off issuing an XIP fetch while indirect_path_busy (or the RX-drain settle guard) is asserted.

BLOCK WISE EXPLANATION

1. apb_slave.v — APB register bank / firmware interface

Standard APB3 FSM (idle/setup/access). Holds the programmer's model: cmd_reg, addr_reg, dummy_reg, len_reg, transfer_config_reg, mode_data_reg, tx_fifo_data, and a self-clearing ctrl_reg[0] start bit. Register writes are blocked while indirect_path_busy or xip_path_busy is high (except write_fifo_data, which is always live so firmware can stream TX bytes). Produces start (rising-edge detect on ctrl_reg[0]) and start_toggle (toggle-flop, crosses to sclk domain via domain_crossing). Read-only status register mirrors {transmission_completed, indirect_path_busy, rx_data_valid}. pslverr is currently tied to 0 — known gap (undefined offsets 0xA–0xF don't fault), deferred pre-handoff.

The firmware has to feed the register bank of APB with the data required for flash communication. For example, Firmware wants to perform “Read Status Register” operation, then it has to write 0x06 into the “cmd_reg” of the APB Register bank, similarly it has feed data to other register and assert the start control signal present in “Control Status” Register.

Note: If Indirect path/XIP path is busy, then even if the firmware tries to write into the APB Register Bank it will be discarded. Monitor the “Transmission Completed” Flag present in APB Register Bank, whenever the flag is asserted, if it read operation, output will be pushed into “read_fifo_dataout” APB Register after the fourth clock cycle.

2. ahb_slave.v — AHB-Lite XIP slave FSM

Single-beat NONSEQ-driven FSM: idle → check_prefetch_buffer → (data_tag_hit on a cache hit, or wait_engine → fetch_wait → fill_prefetch_buffer on a miss). Latches haddr/hsize/hburst/hwrite at address-phase accept since AHB only guarantees them stable that one cycle. Read-only enforcement: any write attempt goes to write_reject and asserts hresp (ERROR). xip_path_busy is asserted only during fetch_wait (i.e., while the shared engine is actually being driven by the XIP register bank) — this is the signal the top-level ownership mux keys off of. wait_engine holds off issuing a fetch while indirect_path_busy |

rx_drain_busy (APB priority). HRDATA mux extracts byte/halfword/word from the 128-bit cache line based on haddr_latched[3:0].

Since XIP Path is read only, if write operation is performed via AHB it will lead to HSLVERR assertion.

Corner case: Suppose prefetch buffer stored 16 bytes of data starting from 0x000000 address of the flash, and then if Indirect mode proceeds to write/erase data of that same address, then prefetch buffer invalidates the data stored by asserting the invalid flash.

3. prefetch_buffer.v — single-line direct-mapped cache

One 16-byte line, tag = addr[23:4]. tag_hit combinational compare. Filled by fill_pulse/fill_addr/fill_data from the XIP engine's completed QIOR. Invalidated by invalidate_pulse, driven from top-level logic that detects an APB write (PP/erase) overlapping the currently cached line's address range — keeps XIP from serving stale data after firmware programs/erases the same region it's currently cached.

4. xip_command_engine.v — autonomous WIP-poll + QIOR sequencer

State machine: idle → issue_rdsr → wait_rdsr → (loop back to issue_rdsr if WIP still set, else) → issue_qior → wait_qior → done. Drives its own register bank (xip_cmd_reg, xip_addr_reg, etc.) which the top-level mux substitutes for the APB registers whenever xip_path_busy is high. Completion is detected by counting rx_byte_ready_xip pulses to the expected length (1 for RDSR, 16 for QIOR) rather than watching transmission_completed, matching the same "byte-count is authoritative" philosophy used elsewhere. MAX_WIP_POLLS (default 65535) bounds the RDSR poll loop and raises timeout_error if WIP never clears — margin against worst-case flash timing is unverified, flag for DV to check against tPP/tSE/tBE.

Note: Suppose Indirect Mode has performed “Block Erase” operation on the Flash, then even after the operation has completed, the flash takes certain amount of time depending on its model (around 50ms) to complete this operation during which **WIP** flag will be high. It is responsibility of the XIP Path to poll for WIP Flag. This is done via issuing command code of “Read Status Register”, in the one byte output received, if **WIP** flag is low, then start reading data from the flash via “QIOR” command, else keep polling “RDSR” until WIP flag is low.

5. domain_crossing.v — pclk → sclk control-register CDC

Toggle-flop CDC for start: a 2-FF dual-rank synchronizer plus a 1-cycle "prev" register produces an XOR-detected edge pulse (start_pulse) in the sclk domain. On that pulse the FSM (idle → latch_inputdata → send_data) latches cmd/addr/dummy/len/mode/transfer_config into the _sync registers and asserts start_out for exactly one sclk cycle. This is the only place control/config registers cross domains — the toggle-flop pattern is required (not a plain 2-FF sync) because start is a single-pclk-cycle pulse and pclk is faster than sclk, so a plain level synchronizer could miss it entirely.

This module receives request from both Indirect and XIP path, manages the hierarchy and send commands to “qspi_flash_controller” module based on priority.

PCLK -> SCLK domain crossing take place in this module. Enable signal based crossing is used. Both the Indirect and XIP path send a start signal via a toggle synchronizer. to this module as a request.

Toggle Synchronizer: The start signal is a pulse in the PCLK domain. Since PCLK and SCLK are asynchronous, a short pulse may not be sampled by the SCLK domain if it occurs entirely between two SCLK sampling edges. Therefore, the pulse is converted into a level change using a T flip-flop. Whenever a start pulse is received, the T flip-flop toggles its output. This toggle is then passed through a multi-flop synchronizer into the SCLK domain. In the SCLK domain, the synchronized toggle is XORed with its previous value. When the toggle

changes, the XOR output becomes logic 1 for one SCLK cycle, generating a start pulse in the SCLK domain.

6. qspi_flash_controller.v — the actual SPI/QSPI protocol engine (sclk domain)

Generic phase sequencer, not a per-opcode case statement: states are opcode → address → mode → dummy → data, and which phases are active/how many IO lines are used is entirely driven by transfer_config_reg[9:0] (see field layout below). Handles 1/2/4-wire shifting for address/mode/data independently, drives cs/si/so/wp/sio3, and produces dout/valid on RX bytes and tx_data_req pulses timed to arrive one cycle ahead of when TX data is actually latched (compensates for the TX FIFO's registered read latency).

The commands to be sent to the flash(cmd_reg, addr_reg, etc) is sent via “domain_crossing” module. If data is to be written into the flash, those data are sent via **TX FIFO** module. If output data is present, then it is sent back to **APB/Prefetch Buffer** via **RX FIFO**.

This module has to be connected to “Behavioral Flash Model”.

7. fifo_write_fsm.v / TX async_fifo / fifo_read_fsm.v — TX path (firmware → flash)

fifo_write_fsm (pclk) two-stage-pulses wen one cycle after tx_fifo_wr_pulse so tx_fifo_data has settled. async_fifo (256-deep, ADDR_WIDTH=8) is the standard dual-clock gray-code-pointer FIFO — write side pclk, read side sclk. fifo_read_fsm (sclk) asserts ren whenever data_direction is write-class, start is high, and byte_req (tx_data_req from the flash controller) fires, counting count_bytes_sent against len_reg.

Whenever firmware pushes a data into “write_fifo_data” Register in the APB Register Bank, that data byte is pushed into the TX FIFO. Since the maximum bytes of data written into the flash is 256, the depth of TX FIFO is chosen to be 256. When Flash Controller starts sending data to the Flash, then during **Data Phase** each byte is read from TX FIFO one by one and sent to the Flash.

8. rx_fifo_write_logic.v / RX async_fifo / rx_fifo_read_logic.v — RX path (flash → firmware/XIP)

rx_fifo_write_logic (sclk) latches dout into the FIFO on valid with a one-cycle wen pulse. async_fifo (32-deep, ADDR_WIDTH=5) — write side sclk, read side pclk. rx_fifo_read_logic (pclk) is where Bug 1 (concurrent APB QOR + AHB XIP corruption) was fixed: a latched drain_pending condition (armed on start_pulse, cleared when count_bytes_read reaches the transfer's own len_reg) drains the FIFO whenever data_direction is read-class and the FIFO isn't empty — this lets RX bytes stream out concurrently with the flash transfer rather than waiting for it to finish. data_direction here must be wired to dc_tcfg_in (the muxed value), not the raw APB transfer_config_reg, or the XIP path's RDSR poll can permanently stall (see notes below). Includes a ren_d pipeline stage so read_data doesn't sample the FIFO's registered output one cycle too early.

At the top level, rx_byte_ready fanout is gated into rx_byte_ready_apb/rx_byte_ready_xip (AND'd with ~xip_path_busy / xip_path_busy respectively) so APB and XIP never see each other's captured bytes on the shared RX chain.

9. qspi_status.v — busy/done generation (sclk domain)

Tracks cs transitions: idle → qspi_operation (cs low) → qspi_done (cs high) → idle. Produces busy/done (immediate, feeds indirect_mode_status_decoder) and done_delayed (4-cycle fixed delay, feeds only rx_fifo_read_logic's qspi_done input — covers the RX write-latch + FIFO-write + gray-pointer-register pipeline

so the FIFO's write pointer is visibly updated on the read side before the drain FSM starts pulling).

10. indirect_mode_status_decoder.v — pclk-domain busy/done for APB

Dual-rank-synchronizes qspi_done from sclk into pclk, edge-detects it, and drives indirect_path_busy/transmission_completed for apb_slave and the top-level arbitration/invalidation logic. State machine: idle → transmission_begun (on start) → transmission_completed_by_qspi (on synchronized done pulse) → idle.

11. async_fifo.v — shared dual-clock FIFO primitive

Parameterized (ADDR_WIDTH), used for both TX (256-deep) and RX (32-deep). Standard Gray-code read/write pointer CDC: binary pointer → Gray-code → dual-rank synchronize into the other domain → Gray-to-binary reconvert (generate-loop XOR chain, covers all bit positions including the MSB). full/empty derived by comparing synchronized pointers.

12. reset_sync.v — per-domain reset synchronizer

Simple 2-FF synchronizer, one instance per clock domain (u_rst_pclk/u_rst_sclk in top). Known open item: presn is not currently synchronized into the async FIFOs themselves (they take the raw/domain-synced reset directly) — flagged, not yet applied.

13. top_cdc_fifo_integrated.v — top-level glue

Instantiates everything above, plus:

The ownership mux (dc_cmd_in, dc_addr_in, ... dc_tcfg_in) that feeds domain_crossing from either APB or XIP registers based on xip_path_busy.

APB write-overlap invalidation logic: latches the address of any write-class APB command (is_write_class_cmd) on its start pulse, and on completion (apb_write_done_pulse) checks whether it overlapped the currently-cached XIP line (addr_overlaps_tag, with erase-size-aware masking for SE/BE) to fire invalidate_pulse into prefetch_buffer.

The RX-drain settle guard (rx_drain_settle / rx_drain_busy): holds ahb_slave off from stealing the shared engine until the RX FIFO has read empty for 3 straight pclk cycles, preventing XIP's own start pulse from resetting count_bytes_read mid-drain of an APB transfer.

REGISTER MAP

1. APB Register Bank

Offset (Hex)	Name	Reset value	Width	Access	Gating Condition	Description
0x0	command_code	0	8	RW	~indirect_path_busy	Opcode byte sent as the QSPI opcode phase
0x1	address_bytes	0	24	RW	~indirect_path_busy	Flash address for address-phase-enabled transactions. Future scope: Four byte address
0x2	dummy_cycles_count	0	5	RW	~indirect_path_busy	Dummy SCLK cycles count required for the particular flash command code
0x3	length_of_data	0	9	RW	~indirect_path_busy	Number of bytes to be written/read from the flash
0x4	control_signals	0	2	W	~indirect_path_busy & ~start_pending & ~xip_path_busy	LSB: Start transmission MSB: Abort transmission (future scope of work)
0x5	write_fifo_data	0	8	W	None — not gated by indirect_path_busy	Bytes of data to be written into the flash has to store into this register one by one.
0x6	read_fifo_data	0	8	R	Read of this offset clears rx_data_valid	During read operation, the flash stores output data in 'rx_fifo', and the byte from the 'rx_fifo' are loaded into this register one by one.
0x7	transfer_configuration_reg	0	10	RW	~indirect_path_busy	Phase-sequencer control bitfield; see transfer configuration bitfield table below. FSM is opcode-blind — entirely driven by this register
0x8	qspi_transfer_status_register	0	3	R	None — read-only status	{transmission_completed, indirect_path_busy, rx_data_valid} —

						packed into prdata[2:0], upper bits 0
0x9	mode_byte	0	8	RW	~indirect_path_bu sy	M7-M0 mode byte for flash transaction. Value depends on the behavioral flash module used.

2. Transfer Config Register of APB Register Bank

Bit Range	Field Name	Values	Description
[9:8]	data_phase_line_width	00 = 1-wire, 01 = 2-wire, 10 = 4-wire	I/O width during the data phase
[7:6]	addr_phase_line_width	00 = 1-wire, 01 = 2-wire, 10 = 4-wire	I/O width during the address/mode/dummy phases
[5]	addr_byte_count	0 = 3-byte address, 1 = 4-byte address	Selects 24-bit vs 32-bit address field width. Note: Current RTL only supports 3 byte address due to conflict with prefetch buffer
[4]	data_direction	0 = read from flash, 1 = write to flash	Direction of the data phase
[3]	addr_phase_enable	0 = skip, 1 = include	Whether an address phase follows the opcode
[2]	data_phase_enable	0 = skip, 1 = include	Whether a data phase is present
[1]	mode_phase_enable	0 = skip, 1 = include	Whether a mode-byte phase follows the address phase
[0]	dummy_phase_enable	0 = skip, 1 = include	Whether a dummy-cycle phase is present

RTL SOURCE

Directory structure:

qspi_flash_controller

- RTL
 - command_code.h
 - reset_sync.v
 - domain_crossing.v
 - async_fifo.v
 - fifo_write_fsm.v
 - fifo_read_fsm.v
 - rx_fifo_write_logic.v
 - rx_fifo_read_logic.v
 - qspi_status.v
 - qspi_flash_controller.v
 - prefetch_buffer.v
 - indirect_mode_status_decoder.v
 - apb_slave.v
 - ahb_slave.v
 - xip_command_engine.v
 - top_module.v

model

- MX25U6432FM2I02.v

tb

- tb.v

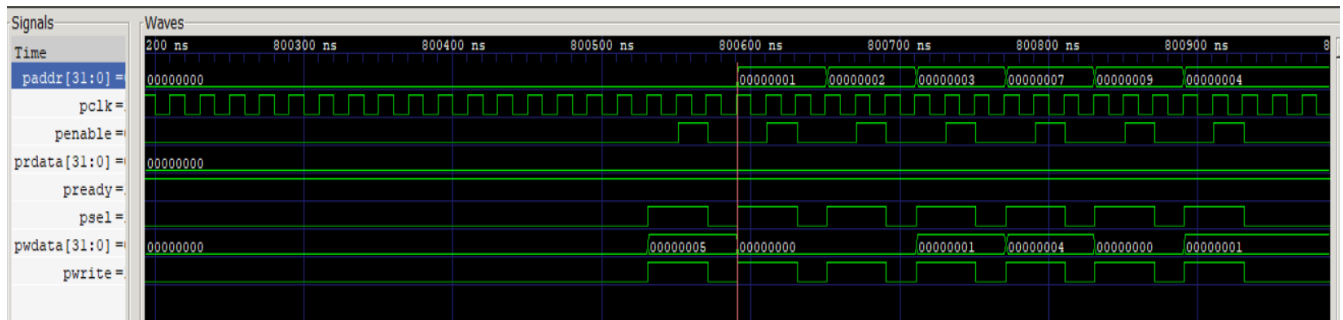
File list:

S.No	File	Description
1	command_code.h	Register offsets, command/opcode definitions, bit-field definitions and fixed XIP presets
2	reset_sync.v	Asynchronous reset assertion and synchronous reset deassertion for each clock domain
3	domain_crossing.v	Toggle-based PCLK-to-SCLK clock-domain crossing for the selected command/register set. Send commands to Flash Controller based on XIP and Indirect Mode request.
4	async_fifo.v	Dual-clock asynchronous FIFO using Gray-coded read/write pointers
5	fifo_write_fsm.v	Generates TX FIFO write control from APB FIFO writes
6	fifo_read_fsm.v	Reads TX FIFO data and supplies it to the QSPI Flash Controller
7	rx_fifo_write_logic.v	Captures valid QSPI RX data and generates RX FIFO write control
8	rx_fifo_read_logic.v	Reads received data from the RX FIFO and generates rx_byte_ready
9	qspi_status.v	Generates QSPI transaction busy, done and delayed-done status
10	qspi_flash_controller.v	Main QSPI serial transaction engine, drives serial pins of behavioral flash model
11	prefetch_buffer.v	One-line XIP prefetch/cache buffer. Each line contains 16 bytes of data. AHB Slave checks this prefetch buffer, if tag is present and is valid, then data is fetch from prefetch buffer, else data for the new address is fetched from the flash and stored back in prefetch buffer. If Indirect mode writes data into a address, and Prefetch Buffer happens to store data of the same address, then that data is invalidated.
12	indirect_mode_status_decoder.v	Generates indirect-path busy and transmission-completed status, which is then sent to PCLK domain via dual rank synchronizer. These signals are used to write into "qspi_transfer_status_register" of the APB Register Bank.
13	apb_slave.v	APB interface and indirect-mode register bank
14	ahb_slave.v	AHB-Lite interface and XIP request handling

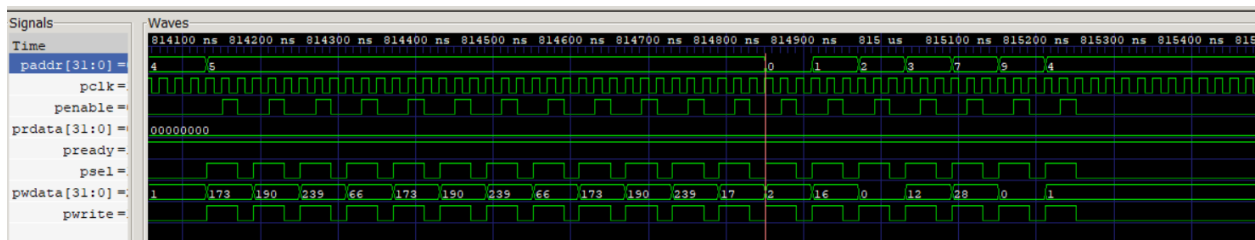
15	xip_command_engine.v	Generates the command/configuration required for an XIP Flash read
16	top_module.v	Top-level integration of APB, AHB/XIP, ownership mux, CDC, FIFOs, QSPI controller and status logic

INTERFACE TIMING DIAGRAM

1. Starting Indirect Mode Transmission for command code 0x05 – Read Status Register

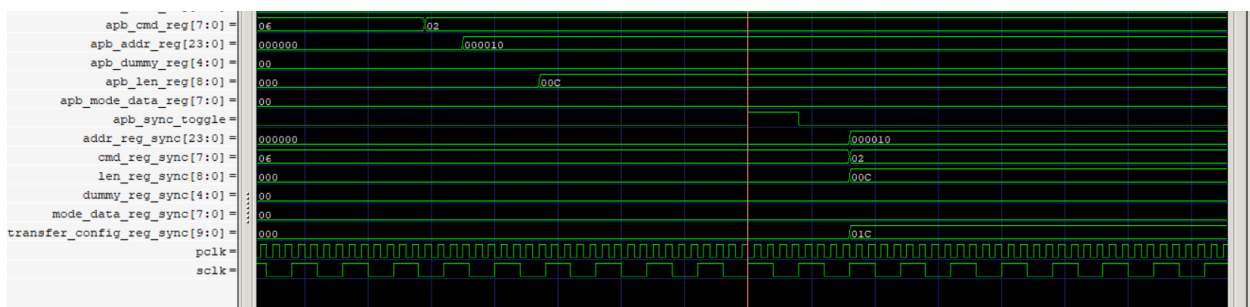


2. Starting Indirect Mode Transmission for command code 0x02 – Page Program



First 12 bytes of data to be written into the flash is written into “write_fifo_dataout” (0x05 Offset) Register of APB Bank. Then the command registers are written and transmission is started at the end. Notice the value of 12 loaded into “len_reg” (0x03 Offset)

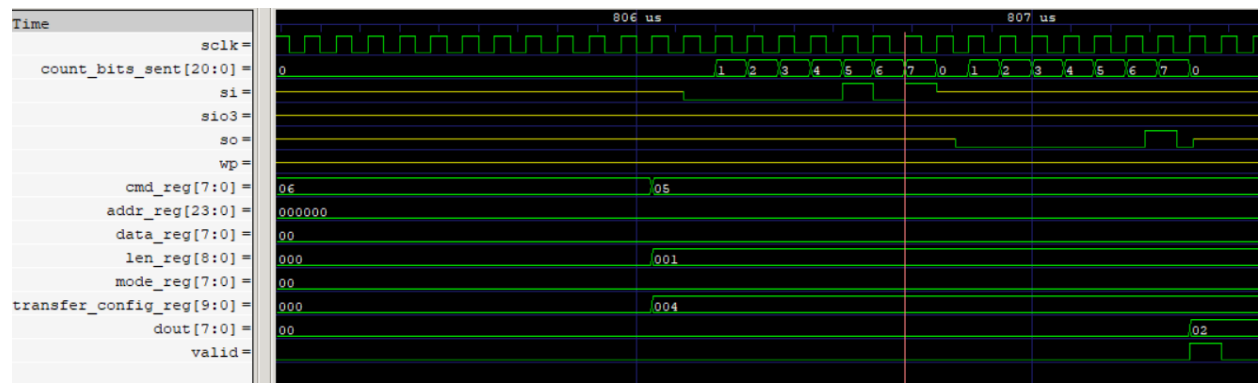
3. Data crossing for Indirect Mode Request



Notice the value of register change every two pclk cycles, this is because Firmware writes data into APB Register Bank one by one. Whenever “domain_crossing” module receives this pulse it latches the commands.

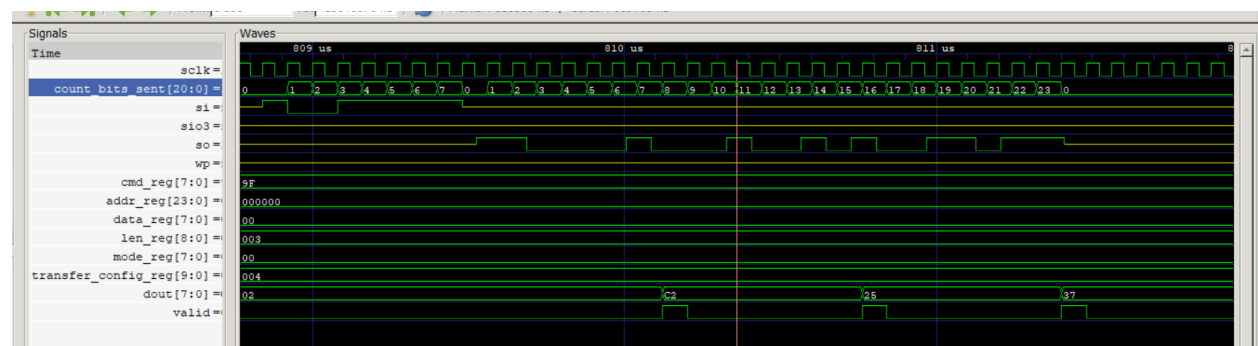
4. QSPI Flash Controller communicating with Behavioral Flash Model

a. Command Code – 0x05 Read Status Register



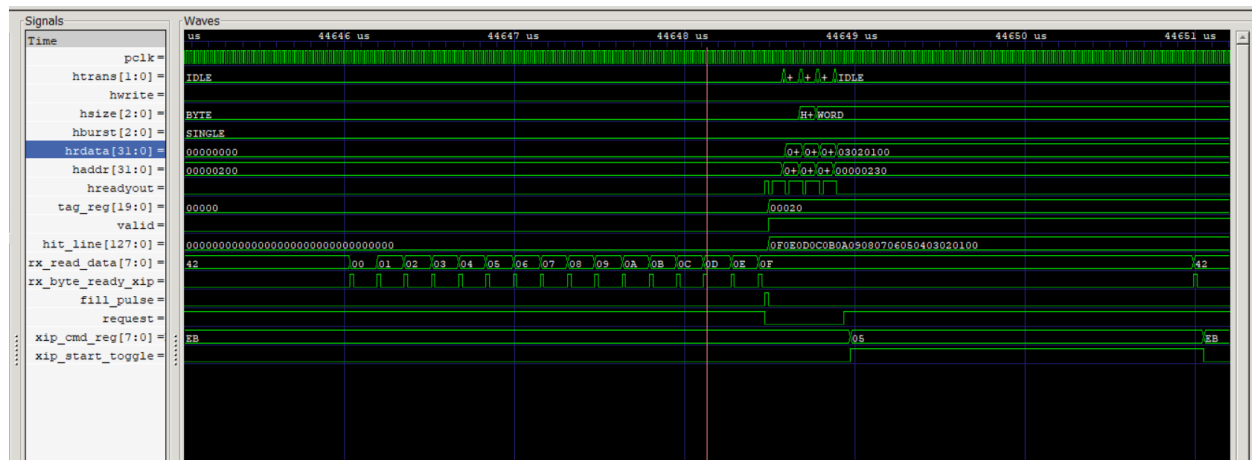
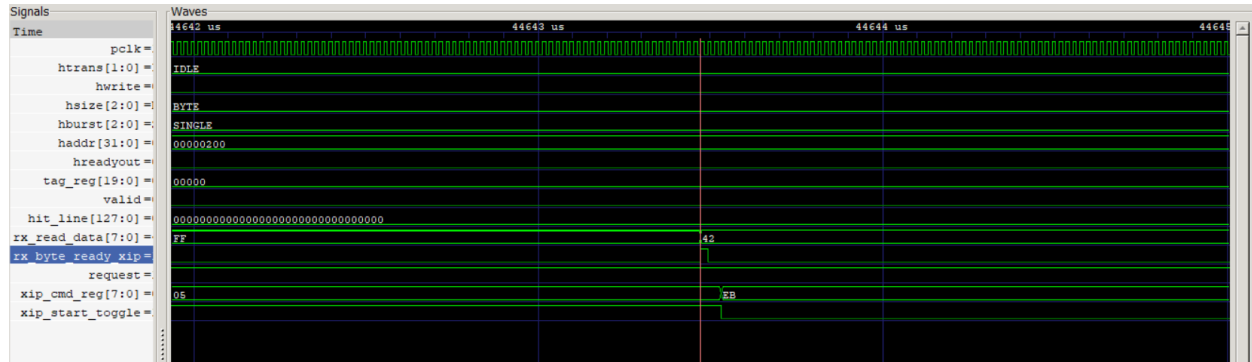
Since it is RDSR Command first opcode/command code of 0x05 is sent via **si** pin to the flash, after the 8 bits are sent, flash sent value of **Status Register** via **so** pin. Once all the 8 bits are received **valid** signal is asserted and data is sent via **dout**. This data is sent to **RX FIFO**.

b. Command Code – 0x9F Read JDEDEC ID



c. Command code – 0x02 Page Program

XIP Path now has to fetch data from the flash via **QIOR**. Before that XIP has to check if **WIP** flag is low. Hence it first sends cmd_reg 0x05 RDSR, and then checks for data received, since output of RDSR command was 0x42 meaning WIP is low, it then performs QIOR.



Once 16 bytes of data is fetched from the flash, fill pulse is sent to prefetch buffer. The prefetch buffer then fills its line data and asserts the **valid flag**. Once the AHB Slave sees this **valid flag**, data of the corresponding address is fetched from the prefetch buffer and sent out via **hrdata**

SELF CHECKING TESTBENCH



tb.v

Note: The failures reported by the self-checking testbench are due to minor errors in the testbench. The RTL functionality is correct, and no corresponding failures are observed in the simulation waveform.

Output Log:

VCD info: dumpfile wave.vcd opened for output.

[0] flash WIP=0 WEL=0 QE=0

[8005200] ---- RDSR1 (post-reset, expect WEL=0/WIP=0) ----

[8032300] PASS : RDSR1 post-reset WIP==0 (expected=0 actual=0)

[8032300] PASS : RDSR1 post-reset WEL==0 (expected=0 actual=0)

[8032300] ---- WREN ----

[8046800] flash WIP=0 WEL=1 QE=0

[8053100] ---- RDSR1 (post-WREN, expect WEL=1) ----

[8080300] PASS : RDSR1 post-WREN WEL==1 (expected=1 actual=1)

[8080300] ---- RDID ----

[8120300] PASS : RDID last byte (memory density) (expected=37 actual=37)

[8120300] ---- WREN (before page program @0x10) ----

[8141100] ---- PAGE PROGRAM (12 bytes @ 0x000010) ----

[8258800] flash WIP=1 WEL=1 QE=0

[12258800] flash WIP=0 WEL=0 QE=0

[13265100] flash[0x10] = ad

[13265100] flash[0x11] = be

[13265100] flash[0x12] = ef

[13265100] flash[0x13] = 42

[13265100] flash[0x14] = ad

[13265100] flash[0x15] = be
[13265100] flash[0x16] = ef
[13265100] flash[0x17] = 42
[13265100] flash[0x18] = ad
[13265100] flash[0x19] = be
[13265100] flash[0x1a] = ef
[13265100] flash[0x1b] = 11
[13265100] PASS : direct readback last byte @0x1B (expected=11 actual=11)
[13265100] ---- WREN (before FAST READ tests) ----
[13279600] flash WIP=0 WEL=1 QE=0
[13285900] ---- FAST READ (12 bytes @ 0x000010, 7 dummy) ----
[13409100] PASS : FAST_READ last byte @0x10 (expected=11 actual=11)
[13526800] flash WIP=1 WEL=1 QE=0
[17526800] flash WIP=0 WEL=0 QE=0
[18533100] ---- DUAL OUTPUT READ (DOR 0x3B, 12 bytes @ 0x000030, 7 dummy) ----
[18617900] PASS : DOR last byte @0x30 (expected=0b actual=0b)
[18617900] ---- DUAL I/O READ (DIO 0xBB, 12 bytes @ 0x000030, 3 dummy) ----
[18689900] PASS : DIO last byte @0x30 (expected=0b actual=0b)
[18689900] ---- WREN (before PP@0x70) ----
[18704400] flash WIP=0 WEL=1 QE=0
[18828400] flash WIP=1 WEL=1 QE=0
[22828400] flash WIP=0 WEL=0 QE=0
[23834700] ---- WREN (before WRSR) ----
[23849200] flash WIP=0 WEL=1 QE=0
[23879600] flash WIP=1 WEL=1 QE=0
[423879600] flash WIP=0 WEL=0 QE=1
[433885900] PASS : WRSR sets QE==1 (expected=1 actual=1)
[433885900] ---- RDSR1 (post-WRSR) ----

[433913100] PASS : R1 post-WRSR QE bit readback ==1 (expected=1 actual=1)

[433913100] ---- QUAD OUTPUT READ (QOR 0x6B, 12 bytes @ 0x000070, 7 dummy) ----

[433978700] PASS : QOR last byte @0x70 (expected=bb actual=bb)

[433978700] ---- QUAD I/O READ (QIOR 0xEB, 12 bytes @ 0x000070, 5 dummy) ----

[433978700] NOTE: mode phase is a no-op here -- top ties

 qspi_flash_controller.mode_reg to a constant 0 placeholder 43397870

 (no APB path for mode_byte yet). 43397870

[434028300] PASS : QIOR last byte @0x70 (expected=bb actual=bb)

[434028300] ---- WREN (before QIPP) ----

[434042800] flash WIP=0 WEL=1 QE=1

[434049100] ---- QUAD INPUT PAGE PROGRAM (QIPP 0x32, 4 bytes @ 0x000100) ----

[434077200] flash WIP=1 WEL=1 QE=1

[438077200] flash WIP=0 WEL=0 QE=1

[439083500] ---- WREN (before 256B page program @0x200) ----

[439098000] flash WIP=0 WEL=1 QE=1

[439104300] ---- PAGE PROGRAM (256 bytes @ 0x000200, incrementing pattern) ----

[440930000] flash WIP=1 WEL=1 QE=1

[444930000] flash WIP=0 WEL=0 QE=1

[445936300] PASS : PP 256B direct readback @0x200 (256 bytes verified, incrementing pattern)

[445936300] ---- WREN (before 256B QUAD OUTPUT READ) ----

[445950800] flash WIP=0 WEL=1 QE=1

[445957100] ---- QUAD OUTPUT READ (QOR 0x6B, 256 bytes @ 0x000200, 7 dummy) ----

[446413100] PASS : OR 256B streamed readback @0x200 (256 bytes verified, incrementing pattern)

[446413100] ===== XIP / AHB PATH TESTS =====

[446413100] ---- [XIP] Cold-miss BYTE read @0x000200 ----

[446485510] PASS : -miss byte read @0x200 HRESP==OK (expected=0 actual=0)

[446485510] PASS : XIP cold-miss byte read @0x200 (expected=00000000 actual=00000000)

[446485510] ---- [XIP] Cache-hit BYTE read @0x000205 ----

[446486510] PASS : e-hit byte read @0x205 HRESP==OK (expected=0 actual=0)

[446486510] PASS : XIP cache-hit byte read @0x205 (expected=00000005 actual=00000005)

[446486510] ---- [XIP] Cache-hit HALFWORD read @0x000208 ----

[446487510] PASS : t halfword read @0x208 HRESP==OK (expected=0 actual=0)

[446487510] PASS : P cache-hit halfword read @0x208 (expected=00000908 actual=00000908)

[446487510] ---- [XIP] Cache-hit WORD read @0x00020C ----

[446488510] PASS : e-hit word read @0x20C HRESP==OK (expected=0 actual=0)

[446488510] PASS : XIP cache-hit word read @0x20C (expected=0f0e0d0c actual=0f0e0d0c)

[446488510] ---- [XIP] Cold-miss WORD read @0x000230 (new line) ----

[446560710] PASS : -miss word read @0x230 HRESP==OK (expected=0 actual=0)

[446560710] PASS : XIP cold-miss word read @0x230 (expected=33323130 actual=33323130)

[446560710] ---- [XIP] Write-overlap invalidation @0x000230 ----

[446560710] PASS : lid before overlapping APB write (expected=1 actual=1)

[446560710] ---- WREN (before overlap PP @0x230) ----

[446726800] flash WIP=1 WEL=1 QE=1

[450726800] flash WIP=0 WEL=0 QE=1

[451733100] PASS : ed after overlapping APB program (expected=0 actual=0)

[451733100] ---- [XIP] Re-read @0x000230 after invalidation (must fetch updated data) ---
-

[451805510] PASS : alidate+refetch @0x230 HRESP==OK (expected=0 actual=0)

[451805510] PASS : after invalidate+refetch @0x230 (expected=23222120 actual=23222120)

[451805510] ---- [XIP] AHB WRITE attempt @0x000200 (expect HRESP=ERROR) ----

[451806310] PASS : IP region rejected (HRESP=ERROR) (expected=1 actual=1)

```

[451806310] PASS : pt left flash contents unchanged (expected=00 actual=00)
[451806310] ---- [XIP] Concurrent APB QOR (256B) + AHB read while indirect_path_busy=1
----
[452261900] FAIL : ill correct under XIP contention -- byte[48] expected=30 actual=20
[452261900] FAIL : ill correct under XIP contention -- byte[49] expected=31 actual=21
[452261900] FAIL : ill correct under XIP contention -- byte[50] expected=32 actual=22
[452261900] FAIL : ill correct under XIP contention -- byte[51] expected=33 actual=23
[452261900] FAIL : ill correct under XIP contention -- byte[52] expected=34 actual=24
[452261900] FAIL : ill correct under XIP contention -- byte[53] expected=35 actual=25
[452261900] FAIL : ill correct under XIP contention -- byte[54] expected=36 actual=26
[452261900] FAIL : ill correct under XIP contention -- byte[55] expected=37 actual=27
[452261900] FAIL : ill correct under XIP contention -- byte[56] expected=38 actual=28
[452261900] FAIL : ill correct under XIP contention -- byte[57] expected=39 actual=29
[452261900] FAIL : ill correct under XIP contention -- byte[58] expected=3a actual=2a
[452261900] FAIL : ill correct under XIP contention -- byte[59] expected=3b actual=2b
[452261900] FAIL : ill correct under XIP contention -- byte[60] expected=3c actual=2c
[452261900] FAIL : ill correct under XIP contention -- byte[61] expected=3d actual=2d
[452261900] FAIL : ill correct under XIP contention -- byte[62] expected=3e actual=2e
[452261900] FAIL : ill correct under XIP contention -- byte[63] expected=3f actual=2f
[452331910] PASS : irect_path_busy @0x280 HRESP==OK (expected=0 actual=0)
[452331910] PASS : behind indirect_path_busy @0x280 (expected=83828180
actual=83828180)
[452331910] ---- [XIP] Non-overlapping APB write must not invalidate cached line @0x280
----
[452331910] ---- WREN (before non-overlap PP @0x300) ----
[452346000] flash WIP=0 WEL=1 QE=1
[452414000] flash WIP=1 WEL=1 QE=1
[456414000] flash WIP=0 WEL=0 QE=1
[457420300] PASS : after non-overlapping APB write (expected=1 actual=1)

```

[457421110] PASS : pping APB write @0x285 HRESP==OK (expected=0 actual=0)

[457421110] PASS : non-overlapping APB write @0x285 (expected=87868584 actual=87868584)

[457421110] ===== AHB NONSEQ -> SEQ TESTS =====

[457421110] ---- [XIP] Legal NONSEQ -> SEQ 4-beat burst @0x200 ----

[457421320] PASS : SEQ burst @0x200 beat0 HRESP==OK (expected=0 actual=0)

[457421320] FAIL : ONSEQ->SEQ burst @0x200 beat0 @0 (expected=03020100 actual=87868584)

[457492910] PASS : SEQ burst @0x200 beat1 HRESP==OK (expected=0 actual=0)

[457492910] PASS : ONSEQ->SEQ burst @0x200 beat1 @4 (expected=07060504 actual=07060504)

[457493120] PASS : SEQ burst @0x200 beat2 HRESP==OK (expected=0 actual=0)

[457493120] FAIL : ONSEQ->SEQ burst @0x200 beat2 @8 (expected=0b0a0908 actual=07060504)

[457493710] PASS : SEQ burst @0x200 beat3 HRESP==OK (expected=0 actual=0)

[457493710] PASS : ONSEQ->SEQ burst @0x200 beat3 @C (expected=0f0e0d0c actual=0f0e0d0c)

[457494110] ---- [XIP] Legal NONSEQ -> SEQ burst crossing 0x20F/0x210 ----

[457494320] PASS : ary burst @0x20C beat0 HRESP==OK (expected=0 actual=0)

[457494320] PASS : Q boundary burst @0x20C beat0 @0 (expected=0f0e0d0c actual=0f0e0d0c)

[457494910] PASS : ary burst @0x20C beat1 HRESP==OK (expected=0 actual=0)

[457494910] FAIL : Q boundary burst @0x20C beat1 @4 (expected=13121110 actual=03020100)

[457566510] PASS : ary burst @0x20C beat2 HRESP==OK (expected=0 actual=0)

[457566510] PASS : Q boundary burst @0x20C beat2 @8 (expected=17161514 actual=17161514)

[457566720] PASS : ary burst @0x20C beat3 HRESP==OK (expected=0 actual=0)

[457566720] FAIL : Q boundary burst @0x20C beat3 @C (expected=1b1a1918 actual=17161514)

[457567110] ---- [XIP] New NONSEQ burst after IDLE ----

[457639110] PASS : after SEQ burst @0x220 HRESP==OK (expected=0 actual=0)

[457639110] PASS : ew NONSEQ after SEQ burst @0x220 (expected=23222120 actual=23222120)

[457639110] ===== XIP CORNER-CASES =====

[457639110] ---- [XIP] Byte read at line boundary offset 15 @0x20F ----

[457711110] PASS : st byte of line 0x200) HRESP==OK (expected=0 actual=0)

[457711110] PASS : @0x20F (last byte of line 0x200) (expected=0000000f actual=0000000f)

[457711110] ---- [XIP] Byte read at next line start @0x210 ----

[457783110] PASS : st byte of line 0x210) HRESP==OK (expected=0 actual=0)

[457783110] PASS : 0x210 (first byte of line 0x210) (expected=00000010 actual=00000010)

[457783110] ---- [XIP] Halfword read @0x210 (aligned) ----

[457784110] PASS : P halfword read @0x210 HRESP==OK (expected=0 actual=0)

[457784110] PASS : XIP halfword read @0x210 (expected=00001110 actual=00001110)

[457784110] ---- [XIP] Word read @0x210 (aligned) ----

[457785110] PASS : XIP word read @0x210 HRESP==OK (expected=0 actual=0)

[457785110] PASS : XIP word read @0x210 (expected=13121110 actual=13121110)

[457785110] ---- [XIP] Byte read @0x215 (inside line 0x210) ----

[457786110] PASS : XIP byte read @0x215 HRESP==OK (expected=0 actual=0)

[457786110] PASS : XIP byte read @0x215 (expected=00000015 actual=00000015)

[457786110] ---- [XIP] Word read @0x21C (offset 12, aligned) ----

[457787110] PASS : XIP word read @0x21C HRESP==OK (expected=0 actual=0)

[457787110] PASS : XIP word read @0x21C (expected=1f1e1d1c actual=1f1e1d1c)

[457787110] ---- [XIP] Cold-miss @0x000300 (new line) ----

[457859110] PASS : XIP cold-miss @0x300 HRESP==OK (expected=0 actual=0)

[457859110] PASS : XIP cold-miss @0x300 (expected=5d5c5b5a actual=5d5c5b5a)

[457859110] ---- [XIP] Cold-miss @0x000400 (new line, evicts 0x300) ----

[457931110] PASS : XIP cold-miss @0x400 HRESP==OK (expected=0 actual=0)

[457931110] PASS : XIP cold-miss @0x400 (expected=ffffffff actual=ffffffff)

[457931110] ---- [XIP] Re-read @0x000300 (must refetch, was evicted) ----

[458003110] PASS : @0x300 after eviction HRESP==OK (expected=0 actual=0)

[458003110] PASS : IP re-read @0x300 after eviction (expected=5d5c5b5a actual=5d5c5b5a)

[458003110] ---- [XIP] Long erase + XIP read (polling test) ----

[458003110] ---- [XIP] Prime line @0x500 (cold miss) ----

[458075110] PASS : XIP cold-miss @0x500 HRESP==OK (expected=0 actual=0)

[458075110] PASS : XIP cold-miss @0x500 (expected=ffffffff actual=ffffffff)

[458075110] ---- WREN (before long SECTOR ERASE @0x1000) ----

[458089200] flash WIP=0 WEL=1 QE=1

[458095500] ---- Start sector erase @0x1000 (WIP will be high) ----

[458100300] ---- [XIP] Read @0x600 during erase (should poll WIP) ----

[458129200] flash WIP=1 WEL=1 QE=1

[758129200] flash WIP=0 WEL=0 QE=1

[758203910] PASS : ng erase (WIP polling) HRESP==OK (expected=0 actual=0)

[758203910] PASS : 0x600 during erase (WIP polling) (expected=ffffffff actual=ffffffff)

[758203910] ---- [XIP] Post-erase, verify that the line @0x600 was fetched correctly ----

[758204910] PASS : after erase completed HRESP==OK (expected=0 actual=0)

[758204910] PASS : ead @0x600 after erase completed (expected=ffffffff actual=ffffffff)

[758204910] ===== END XIP CORNER-CASES =====

[758204910] ---- WREN (before SECTOR ERASE @0x700) ----

[758219600] flash WIP=0 WEL=1 QE=1

[758225900] ---- SECTOR ERASE (SE 0x20) @ 0x000700 ----

[758259600] flash WIP=1 WEL=1 QE=1

[1058259600] flash WIP=0 WEL=0 QE=1

[1058265900] ---- QUAD OUTPUT READ post-SE (QOR 0x6B, 12 bytes @ 0x000070, 7 dummy) ----

[1058331500] PASS : E erased region reads 0xFF @0x70 (expected=ff actual=ff)

[1058331500] ---- WREN (before BLOCK ERASE) ----

[1058346000] flash WIP=0 WEL=1 QE=1

[1058352300] ---- BLOCK ERASE (BE 0xD8) @ 0x000000 ----

[1058386000] flash WIP=1 WEL=1 QE=1

[4058386000] flash WIP=0 WEL=0 QE=1

[4158392300] ---- QUAD OUTPUT READ post-BE (QOR 0x6B, 12 bytes @ 0x000030, 7 dummy) ----

[4158457900] PASS : E erased region reads 0xFF @0x30 (expected=ff actual=ff)

[4158457900] ---- RDSR2 (0x35) ----

[4158457900] NOTE: this opcode is cmdEQIO on the flash model, not RDSR2 --

issuing it switches the model into QPI mode. 415845790

[4158478700] PASS : QIO switches flash into QPI mode (expected=1 actual=1)

=====

[4158478700] TESTBENCH RESULT: 5/81 CHECKS FAILED

=====

[4158478700] ---- Test sequence complete ----

tb.v:930: \$finish called at 4158488700 (100ps)

CLOCK AND RESET REQUIREMENTS

Clocks

- hclk and pclk are tied together at 50 MHz in this design - a single unified AMBA-side domain by deliberate architectural choice (avoids unguarded CDC crossings inside the APB/AHB arbitration logic).
- sclk = 12.5 MHz, i.e. pclk/sclk = 4:1 in the current bring-up configuration. This ratio is not enforced in RTL - the CDC structures (toggle+3FF, 2FF, gray-coded FIFO pointers) are correct for any ratio, but synchronizer latency margins (e.g. the RX done_delayed 4-cycle pipeline covering the write-latch+FIFO+gray-ptr chain) were sized assuming sclk is meaningfully slower than pclk. Running sclk close to or faster than pclk has not been verified and should be treated as unsupported until re-characterized.
- No clock-divider register exists (see Known Limitations). SCLK frequency is fixed by whatever is fed to the sclk port; there is no runtime-programmable divide.
- CPOL/CPHA are fixed by construction (SI/SO driven combinationally from qspi_flash_controller's registered shift state on sclk edges) - not programmable, no register exists for mode selection.

Reset

- Single active-low async presetn at the top level.
- Synchronized independently per domain via two reset_sync.v instances (u_rst_pclk, u_rst_sclk): 2-FF async-assert / synchronous-deassert, producing presetn_pclk and presetn_sclk.
- No required sequencing between the two domain resets - both reset_sync instances free-run off the same async presetn and their own domain clock; deassert timing between presetn_pclk and presetn_sclk is not synchronized to each other and should not be assumed to align to a particular edge relationship.
- Flash model requirement: MX25U6432FM2I02.v will not accept the first command until tVSL = 800 microseconds after power-up/reset (modeled explicitly in the behavioral model's initial block). Any testbench or bring-up sequence must hold off the first APB/AHB transaction until this window has elapsed after presetn deassertion, or the model will flag an error and refuse the command.
- Reset during an in-flight indirect-mode or XIP transaction has not been systematically tested - see Verification Focus Areas.

CDC INVENTORY

S.No	Signal(s)	Direction	Synchronizing Structure	Location
1	start (indirect-mode kickoff)	PCLK → SCLK	Toggle flop (start_toggle) + 3-FF synchronizer chain + XOR edge detector	domain_crossing.v
2	xip_start_toggle (XIP-owned start, muxed with #1 before crossing)	PCLK → SCLK	Same toggle + 3FF + XOR structure in domain_crossing.v (mux happens before the crossing, in top_cdc_fifo_integrated.v's ownership mux)	domain_crossing.v
3	qspi_done (transaction-complete pulse)	SCLK → PCLK	Plain 2-FF synchronizer + edge detect	indirect_mode_status_decoder.v
4	qspi_busy	SCLK → PCLK	Plain 2-FF synchronizer	apb_slave.v (sync1 / qspi_busy_sync)
5	TX FIFO write pointer (firmware preload data)	PCLK → SCLK	Gray-coded pointer, dual-rank synchronized (async_fifo.v, write_pointer_handler / read_pointer_handler)	async_fifo #(.ADDR_WIDTH(8)) instance (tx_fifo_module)
6	RX FIFO write pointer (captured Flash read data)	SCLK → PCLK	Gray-coded pointer, dual-rank synchronized	async_fifo #(.ADDR_WIDTH(5)) instance (rx_fifo_module)
7	presetn (async reset)	Async → PCLK	2-FF async-assert/sync-deassert reset synchronizer	reset_sync.v (u_rst_pclk)
8	presetn (async reset)	Async → SCLK	2-FF async-assert/sync-deassert reset synchronizer	reset_sync.v (u_rst_sclk)

KNOWN LIMITATIONS AND DELIBERATELY UNIMPLEMENTED FEATURES

1. **Dummy cycle count has to be one less than the value mentioned in the flash specification.** The RTL currently interprets the dummy-cycle configuration value as one less than the dummy-cycle count specified in the flash datasheet. This is a known RTL limitation. Correcting the interpretation would have downstream impact on the existing command configuration and verification environment. Therefore, the current implementation and verification use the datasheet-specified dummy-cycle count decremented by one when programming the corresponding RTL configuration field. A future RTL revision should correct this offset and update the dependent configurations and tests accordingly.
2. No clock-divider register. SCLK frequency is fixed by the input clock; no runtime-programmable divide ratio exists.
3. CPOL/CPHA are fixed, not programmable. No mode-select register exists.
4. write_fifo_data (offset 0x5) is not gated by indirect_path_busy, unlike every other writable register in the bank. A firmware write here mid-transaction is accepted unconditionally. Given the TX FIFO's single-shot preload model (no mid-burst refeed), this is currently harmless in the intended usage pattern (preload fully before issuing start), but it is an inconsistency worth documenting rather than leaving implicit.
5. Register writes during indirect_path_busy are silently dropped, not flagged. pready is unconditionally 1 and pslverr is tied to 0 in every state; a write to a busy-gated register (command_code, address_bytes, etc.) while indirect_path_busy is high completes the APB handshake normally with no error indication to firmware. Two remediation options were discussed and deliberately deferred pre-handoff: stall pready (correct but risks very long wait-states during e.g. tBE=300ms block erase) or assert pslverr on the gated cycle (lower-risk, no bus-timing change). Left as-is for this handoff.
6. HRESP ERROR response is a single-cycle placeholder, not the full 2-phase HREADYOUT sequence the AHB-Lite spec requires for an error response. Flagged in RTL comments as needing verification against ARM_IHI0033A before signoff.

7. HBURST/hburst_latched is latched but functionally unused - reserved for a possible future multi-line prefetch-ahead stretch goal, not implemented.
8. xip_command_engine's MAX_WIP_POLLS default (65535) has not been numerically verified against worst-case flash erase timing. If a genuine block/chip erase ($t_{BE} = 300\text{ms}$, t_{CE} up to $\sim 18\text{s}$ per the Macronix datasheet) is in progress when an XIP fetch targets an uncached line, the RDSR-poll loop must survive polling for the entire erase duration without hitting MAX_WIP_POLLS and returning a spurious timeout_error. This has not been checked against the real poll-loop period; recommend calculating actual poll-to-poll latency and confirming $65535 * (\text{poll period})$ safely exceeds t_{CE} before relying on this in real bring-up.

SUGGESTED VERIFICATION FOCUS AREAS

1. Back-to-back start pulses faster than the toggle synchronizer can settle. The toggle+3FF+XOR structure in domain_crossing.v needs roughly 3 sclk cycles to register a transition. If firmware (or, more realistically, the XIP engine issuing RDSR then immediately QIOR) asserts a second start_toggle transition before the first has propagated through all 3 flops, the second pulse could be missed or coalesced. Given sclk is 4x slower than pclk, this is a real timing corner, not theoretical - worth a directed test issuing start pulses at the minimum legal pclk spacing.
2. RX FIFO wraparound during a full 256-byte QOR streamed through a 32-entry FIFO. The concurrent-drain design means the RX FIFO should never need to hold more than a few bytes at once, but any hiccup in drain_pending's edge cases (e.g. a transaction that finishes exactly as the FIFO is momentarily full) could silently drop or duplicate a byte. Already partially covered by Bug 1's fix, but worth directed tests at the exact FIFO-full boundary, not just nominal-rate draining.
3. XIP arbitration race at the exact cycle indirect_path_busy (or rx_drain_busy) deasserts. ahb_slave's wait_engine polls (indirect_path_busy | rx_drain_busy) every pclk cycle; a test that starts an AHB fetch request in the exact cycle both conditions clear (rather than well before or well after) exercises the arbitration boundary condition most likely to hide an off-by-one.
4. Reset asserted mid-transaction, particularly mid-quad-read or mid-erase. Neither tb_5.v nor the bring-up testbench appears to exercise this. Given presetn_pclk/presetn_sclk deassert independently (no cross-domain reset sequencing), a reset landing asymmetrically between domains while a CDC handshake is in flight (e.g. mid toggle-sync) is a plausible way to wedge a state machine in a stuck state.
5. mode_byte with a Fxh top nibble. Firmware is expected to avoid this to prevent the flash model entering continuous-read mode, but nothing in RTL prevents it. Worth a negative test confirming the design's actual behavior (not just documented "don't do this") if that mode byte is written.
6. dummy_reg = 0 boundary (zero dummy cycles requested on a command that normally needs some) and len_reg = 0 on a data-phase-enabled command - both are representable in the register width but may not be meaningfully supported by the phase-sequencer FSM; confirm the FSM doesn't hang or misbehave rather than just cleanly rejecting/ignoring these.

7. WEL race: a PP/SE/BE/WRSR issued without a preceding WREN in the same transaction sequence (firmware forgets to re-issue WREN after the flash model auto-clears WEL). Confirm the failure mode is clean (op silently ignored per flash model behavior) rather than the controller itself getting into an inconsistent state.
8. Arbitration starvation: can APB, by continuously issuing back-to-back indirect-mode transactions, starve the XIP path indefinitely? wait_engine as written has no fairness/round-robin mechanism - "APB wins on conflict" is correct per spec, but there is no upper bound in RTL preventing genuine starvation if APB traffic is dense enough. Worth characterizing rather than assuming it's a non-issue.
9. MAX_WIP_POLLS vs. real erase timing (see Known Limitations #9) - a directed test that starts a chip erase or block erase and then issues an XIP fetch to an unrelated line, holding it there for the actual worst-case erase duration, would directly validate whether the timeout margin is adequate.
10. Prefetch cache single-line thrash: an AHB access pattern that alternates between two different 16-byte lines on every access (e.g. word A at line X, word B at line Y, repeat) forces a full RDSR+QIOR re-fetch on every single access. Confirm this doesn't expose any state left over from the previous fetch (stale fill_addr, stale byte counters, etc.) rather than just checking it's slow.